

FortSight AI

Anti-Gravity Build Prompt

Real-Time Terrain-Aware Fort Visibility & Spatial Intelligence System

A step-by-step guide extracted from comprehensive requirements.
Build phase by phase. Do NOT build everything at once.

Project	FortSight AI
MVP Region	Maharashtra, India
Core Feature	Terrain-aware fort visibility from user location
Build Strategy	15 sequential phases
Start Date	When you're ready

Table of Contents

1. Executive Summary
2. Problem & Solution
3. MVP Definition
4. Technology Stack
5. Architecture Overview
6. Database Schema Summary
7. Core Modules
8. 15-Phase Build Plan
9. Key Algorithms
10. Critical Success Factors
11. Common Pitfalls
12. Deployment Checklist

1. Executive Summary

Problem: When a trekker stands at a specific location with a compass, they wonder: *Which forts can I actually see from here?* Existing map apps show forts on a map but don't account for terrain obstruction or elevation.

Solution: FortSight AI combines GPS location, device orientation, Digital Elevation Models (DEM), and terrain-aware line-of-sight analysis to determine exactly which forts are visible from the user's current location and viewpoint.

Differentiator: NOT a generic map app. This is terrain-aware, bearing-aware, and orientation-aware visibility calculation with historical fort context.

2. MVP Definition

Minimum Viable Product Scope:

- User opens app on mobile device
- System captures GPS coordinates, elevation, compass heading
- App displays interactive map with current location + Maharashtra forts
- User can click 'What forts can I see?'
- System queries nearby forts (PostGIS spatial filter)
- System retrieves DEM elevation data for user location & forts
- System performs terrain-aware line-of-sight (NOT fake visibility score)
- System returns visible/blocked/uncertain status for each fort
- Map highlights visible forts, dims blocked ones
- User clicks a fort to see: name, distance, bearing, direction, visibility reason

What MVP Does NOT Include:

- RAG chatbot (Phase 12)
- Fort internal structure maps (Phase 10)
- Fort-to-fort visibility network graph (Phase 11)
- Image recognition ML (Phase 13)
- Advanced routing algorithms
- Real-time multiplayer features
- Social features or user accounts (optional auth only)

3. Technology Stack

Layer	Technology	Why
Frontend	React + TypeScript + Vite	Modern, type-safe, fast dev experience
Styling	Tailwind CSS + shadcn/ui	Rapid, consistent UI development
Mapping	Leaflet (or MapLibre GL JS)	Open-source, performant map rendering
State	React Query + Zustand	Async data + global state management
Backend	FastAPI + Python 3.10+	Type hints, async, rapid API development
Database	PostgreSQL + PostGIS	Spatial queries for forts & visibility
Vector DB	pgvector (in PostgreSQL)	Embeddings for RAG chatbot (Phase 12)
GIS Tools	Rasterio + GDAL + GeoPandas	DEM processing, line-of-sight calc
Terrain	SRTM / Copernicus DEM	Free, global elevation data
Deployment	Docker + Docker Compose	Reproducible environments
Server	Nginx + FastAPI + PostgreSQL	Production-grade stack

4. Architecture Overview

System Components:

Frontend (React + Leaflet) — User interface. Captures GPS, heading, displays map.

↓ REST API

FastAPI Backend — Core business logic.

■■■ Auth Module — User authentication (optional)

■■■ Fort Module — Fort CRUD & queries

■■■ Location Module — User location & validation

■■■ Terrain Module — DEM loading & elevation queries

■■■ Visibility Engine — Line-of-sight calculations

■■■ Direction Engine — Bearing & FOV calculations

■■■ RAG Module (Phase 12) — Chatbot & embeddings

■■■ Route Engine — Trekking route suggestions

↓ Database

PostgreSQL + PostGIS — Spatial queries, fort data, terrain metadata

■■■ forts table (geometry, elevation)

■■■ fort_viewpoints table

■■■ fort_structures table

■■■ fort_trails table

■■■ terrain_tiles metadata

■■■ visibility_cache (for performance)

■■■ pgvector embeddings (RAG)

DEM Data (Rasterio) → Cached locally → Queried for elevation → Used in line-of-sight

5. Database Schema (Simplified)

Table	Key Columns
users	id, email, created_at
forts	id, name, marathi_name, geometry(Point,4326), elevation, district, history
fort_viewpoints	id, fort_id, name, type, geometry(Point), elevation
fort_structures	id, fort_id, name, type (machi/bastion/gate), geometry(Point)
fort_trails	id, fort_id, name, difficulty, geometry(LineString), distance
fort_connections	source_fort_id, target_fort_id, distance, bearing, visibility_status
terrain_tiles	id, geom_bbox, dem_path, resolution, created_at
visibility_cache	id, observer_fort_id, target_fort_id, result_json, ttl
historical_documents	id, fort_id, title, content, source
document_chunks	id, doc_id, chunk_text, embedding (pgvector)
chat_sessions	id, user_id, created_at
chat_messages	id, session_id, role, content, created_at

Key Indexes: PostGIS GIST on geometry columns, B-tree on fort_id, btree on terrain_tiles bbox

6. Core Modules

Terrain Module

Load DEM tiles, query elevation at (lat, lon), cache results. Do NOT download entire DEM on startup.

Visibility Engine

Line-of-sight calculation: observer → target, sample terrain between them, detect obstruction. Return VISIBLE/BLOCKED/UNCERTAIN with score.

Direction Engine

Convert bearing (0-360°) → cardinal direction (N/NE/E/etc). Calculate relative angle to device heading. Determine if fort in FOV.

Visibility API

GET /api/v1/visibility/from-location?lat=X&lon=Y&heading=Z&fov=60&radius=50 → Return nearby visible forts ranked by visibility score.

Fort Module

CRUD for forts, viewpoints, structures. Query by location, district, difficulty.

RAG Module (Phase 12)

Embed fort documents → pgvector. On chat query, retrieve relevant docs + call visibility API for live queries.

7. 15-Phase Build Plan

Key Rule: DO NOT skip setup phases. DO NOT build UI before backend is tested.

Phase	Component	Deliverable
Phase 0	Feasibility & Architecture	✓ You are here
Phase 1	Repository & Docker Setup	Set up monorepo, docker-compose, .env files
Phase 2	PostgreSQL + PostGIS Schema	Create migrations, tables, indexes
Phase 3	Fort Data Ingestion	Import Maharashtra forts from CSV/GeoJSON
Phase 4	FastAPI Core Setup	Pydantic schemas, dependency injection, error handling
Phase 5	Terrain Engine	DEM ingestion, elevation queries, caching strategy
Phase 6	Line-of-Sight Algorithm	Implement visibility calculation, unit tests
Phase 7	Visibility API Endpoints	GET /visibility/from-location, test locally
Phase 8	React Map UI	Leaflet integration, show forts, styling
Phase 9	GPS & Heading	Capture device location & orientation
Phase 10	Fort Details & Internal Maps	Fort detail page, viewpoint map
Phase 11	Fort-to-Fort Visibility Graph	Calculate network, store in DB
Phase 12	RAG Chatbot	Embed docs, build chat API, integrate with visibility
Phase 13	ML Optional	Image recognition (if resources allow)
Phase 14-15	Testing & Deployment	Pytest, Vitest, Docker production build

8. Key Algorithms

A. Line-of-Sight Visibility

Input: observer(lat, lon, elev, height), target(lat, lon, elev, height), DEM

Process:

1. Calculate geodesic distance & bearing (using geopy or pyproj)
2. Generate 50-100 sample points along line from observer to target
3. Query DEM elevation at each sample point
4. Calculate sightline elevation: $\text{observer_height} + (\text{distance_ratio} \times (\text{target_height} - \text{observer_height}))$
5. Compare sightline vs terrain_elevation at each point
6. If terrain > sightline at any point: BLOCKED
7. If clear line: VISIBLE
8. If marginal (close to sightline): UNCERTAIN

Output: {status, distance, bearing, max_obstruction, visibility_score}

Score Calculation: $(\text{samples_clear} / \text{total_samples}) \times \text{elevation_bonus} \times \text{distance_factor}$

B. Bearing to Cardinal Direction

Input: bearing (0-360°)

Logic:

0-22.5° → N | 22.5-67.5° → NE | 67.5-112.5° → E | ... | 337.5-360° → N

Field of View (FOV) Filter:

If bearing is within [heading - FOV/2, heading + FOV/2] → fort in view

Output: direction, relative_angle_to_heading, in_fov

C. Spatial Query Optimization

Problem: Cannot run expensive DEM queries on 1000+ forts.

Solution:

1. Use PostGIS to find forts within radius_km using spatial index
2. Filter by elevation difference (±500m max)
3. Calculate bearing for each candidate (fast)
4. Filter by FOV (no DEM needed)
5. Only then run expensive DEM query on ~20 final candidates

Result: Reduced DEM queries from $O(n)$ to $O(\log n)$

9. Critical Success Factors

- **Accurate DEM:** Elevation data MUST be correct. Use SRTM or Copernicus. Validate against known landmarks.
- **Efficient Spatial Queries:** PostGIS indexes are essential. Test query performance with 100+ forts.
- **Clear API Contracts:** Frontend & backend must agree on exact data formats. Use OpenAPI.
- **Reproducible DEM Loading:** Do NOT download DEM on app startup. Provide explicit import script.
- **Deterministic Visibility:** NO fake/arbitrary visibility scores. Every score must be mathematically explainable.
- **Mobile First:** Trekkers use mobile. Test on actual device. Optimize GPS polling.
- **Data Attribution:** Every fort, trail, & document must have source attribution.
- **Testing:** Unit test all calculations. Integration test API endpoints. E2E test full user flow.

10. Common Pitfalls to Avoid

1. Building entire UI before backend visibility API works
2. Hard-coding terrain/fort data instead of using real DEM & database
3. Using arbitrary visibility scores (e.g., random 0-100) instead of mathematically sound scores
4. Forgetting to index PostGIS geometry columns
5. Loading entire DEM into memory on startup (causes 3GB+ memory use)
6. Not validating fort coordinates before storage
7. Trying to build RAG chatbot before core visibility works
8. Not setting up proper .env file (exposing API keys in code)
9. Skipping database migrations (leads to schema drift)
10. Testing only on desktop; ignoring mobile GPS/orientation issues

11. Deployment Checklist

- ✓ All .env secrets in environment variables, not in code
- ✓ Database migrations applied to production DB
- ✓ PostGIS spatial indexes created & analyzed
- ✓ DEM tiles cached locally or in object storage (S3/GCS)
- ✓ Nginx reverse proxy configured for CORS
- ✓ FastAPI running behind Gunicorn with 4+ workers
- ✓ PostgreSQL backups scheduled
- ✓ React build optimized (tree-shaking, code splitting)
- ✓ Logging configured (stdout for container, external for monitoring)
- ✓ Rate limiting enabled on /chat endpoint
- ✓ HTTPS/TLS enforced
- ✓ GitHub Actions CI/CD pipeline set up
- ✓ Docker images tagged with version
- ✓ Load testing passed (1000+ requests/min on visibility API)
- ✓ Mobile testing on 3+ devices/browsers

12. Important Reminders

DO NOT try to build everything at once. The monolith will become unmaintainable.

DO wait for confirmation after each phase before moving to the next.

DO test each component in isolation before integration.

DO use real data: real DEM, real fort coordinates, real elevation.

DO document API contracts & database schema changes.

DO commit frequently. Push to GitHub after each working phase.

This system is production-ready by design. Every component is built with scalability, maintainability, & testability in mind. Do not add "quick hacks" that will bite you later.